

NAME:Shrutha

ROLL NO.: 241801265

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resource matrix, and the Available resource vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format:

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the number of resource types.
- The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
- This is followed by the Maximum Demand (Max) matrix with the same dimensions.
- The final line contains the Available resource vector, where each value represents the number of currently available instances of each resource type.

Select language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int n, r;
6
7     printf("Enter number of processes: \n");
8     scanf("%d", &n);
9     printf("Enter number of resource types: \n");
10    scanf("%d", &r);
11
12    int alloc[n][r], max[n][r], need[n][r], avail[r];
13    int safesquence[n];
14    bool finish[n];
15
16    printf("Enter Allocation Matrix:\n");
17    for (int i = 0; i < n; i++)
18        for (int j = 0; j < r; j++)
```

Text Cases  [Run Sample Cases](#) [View All Codes](#)

Custom Test Case

> Custom Test

Sample Test Case

Q1

Save

Select Language: C (GCC 9.2.0)

```
19    scanf("%d", &alloc[i][j]);
20
21    printf("Enter Max Matrix:\n");
22    for (int i = 0; i < n; i++)
23        for (int j = 0; j < r; j++) {
24            scanf("%d", &max[i][j]);
25            need[i][j] = max[i][j] - alloc[i][j];
26        }
27
28    printf("Enter Available Resources: \n");
29    for (int i = 0; i < r; i++)
30        scanf("%d", &avail[i]);
31
32    for (int i = 0; i < n; i++) finish[i] = false;
33
34    int work[r];
35    for (int i = 0; i < r; i++) work[i] = avail[i];
36
37    int count = 0;
38    while (count < n) {
39        bool found = false;
40        for (int p = 0; p < n; p++) {
41            if (!finish[p]) {
42                bool canallocate = true;
```

Select Language: C (GCC 9.2.0)

```
43         for (int j = 0; j < r; j++) {
44             if (need[p][j] > work[j]) {
45                 canAllocate = false;
46                 break;
47             }
48         }
49
50         if (canAllocate) {
51             for (int k = 0; k < r; k++)
52                 work[k] += alloc[p][k];
53
54             safeSequence[count++] = p;
55             finish[p] = true;
56             found = true;
57         }
58     }
59 }
60
61 if (!found) break;
62 }
63
64 if (count < n) {
65     printf("System is NOT in a safe state (Deadlock may occur).\n");
66 } else {
67     printf("System is in a SAFE STATE.\n");
```

```
62     }
63
64     if (count < n) {
65         printf("System is NOT in a safe state (Deadlock may occur).\n");
66     } else {
67         printf("System is in a SAFE STATE.\n");
68         printf("Safe Sequence: ");
69         for (int i = 0; i < n; i++)
70             printf("%d ", safeSequence[i]);
71         printf("\n");
72     }
73
74     return 0;
75 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Input:

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output:

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output:

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```